

3.0 - SEQUENCE AND GRAPH-BASED NNs

CHAPTER INDEX

- ☐ [INTRODUCTION](#)
- ☐ [RECURRENT NEURAL NETWORKS](#)
 - [MEMORY OF STATE](#)
 - [UNFOLDING IN TIME](#)
 - [DEEP RNNs](#)
 - [LONG SHORT-TERM MEMORY](#)
 - [Bi-LSTM](#)
 - [GRU](#)
- ☐ [TEMPORAL CONVOLUTIONAL NETWORKS](#)
 - [DILATION FACTOR](#)
 - [CAUSAL CONVOLUTIONS, PADDING AND NETWORK ARCHITECTURE](#)
 - [TIME SERIES FORECASTING](#)
 - [COMPARISON OF TCNs AND LSTMs](#)
- ☐ [GRAPH CONVOLUTIONAL NETWORKS](#)
 - [GRAPHS REVISION](#)
 - [GRAPH SIGNALS AND GRAPH SHIFT OPERATORS](#)
 - [GRAPH CONVOLUTIONS](#)
 - [GRAPH PERCEPTRON AND GCNs](#)
 - [TRAINING](#)
 - [SOME PRACTICAL EXAMPLES](#)

INTRODUCTION

Throughout the previous chapters we have covered neural network **architectures** designed to process **inputs of fixed dimension**; particularly **vectors** and **tensors**. Both *multilayer perceptrons* and *convolutional networks* expected **inputs of predetermined size** and processed it to transform it into an **output, again of fixed size**.

We've seen **how powerful** these networks **can be**, but nonetheless the underlying **constraint** of having **inputs** and **outputs of fixed dimensions** is a **huge limitation** for a wide variety of applications where data does **not conform** to this **rigid structure**. For example, consider the following:

☰ VARIABLE-SIZE DATA

- **TEXT** - A **sentence** may have 5, 20, or even 30+ **words**. This means that tasks like text classification **naturally** have to deal with **variable-length sequences**.

- **TIME SERIES** - To deal with **stock prices, sensor readings, signals** (e.g. audio, but also medical signals) we not only have to deal with the *variable length* of the inputs, but we also necessarily **have to consider the evolution over time**, as each **observation** is **useless on its own**.
- **GRAPHS** - In many problems, **relationships** between **entities** can be just as **important as entities themselves**; think of **molecular structures, social networks** or **traffic monitoring**. In all of these cases, data can be represented with **graphs**, whose **structure** may be **arbitrary**.

Thus, this chapter aims to present **three architectures** to deal with each of the **data types** listed above.

We will start off with **Recurrent Neural Networks (RNNs)** and see how we can **process sequences**.

Then, we will present an *alternative approach*: **Temporal Convolutional Networks (TCNs)**, which bring some **computational advantages** over RNNs, other than being **particularly suited** to deal with **time series**.

Finally, we will go over **Graph Convolutional Networks (GCNs)** to see how the **convolution operation** can be **generalised** to **irregular graph structures** to **aggregate information** from each node's **neighbours**.

Before diving into these models, we need to clarify one more thing: **depending on the application**, either the **input**, the **output**, or **both** can be of **variable dimension**.

We thus differentiate between the following **classes of tasks**:

DEFINITION

- **MANY-TO-ONE** - The **input** has **variable length**, while the **output** is **fixed**. An example of such a task is *sentiment analysis*, where the *input* is a *sentence* (e.g. a product review) and the *output* is a *continuous value* that expresses the *sentiment*, or *judgement* (positive/negative).
- **ONE-TO-MANY** - The **input** is **fixed**, while the **output** has **variable length**. An example is given by *image captioning*, where the input is an image, and the *output* is *sentence describing it* (whose length may vary).
- **MANY-TO-MANY** - **Both input and output** are **variable in size**. An important problem that falls into this category is **language translation**, where both the input and output sentences may comprise a variable amount of words.

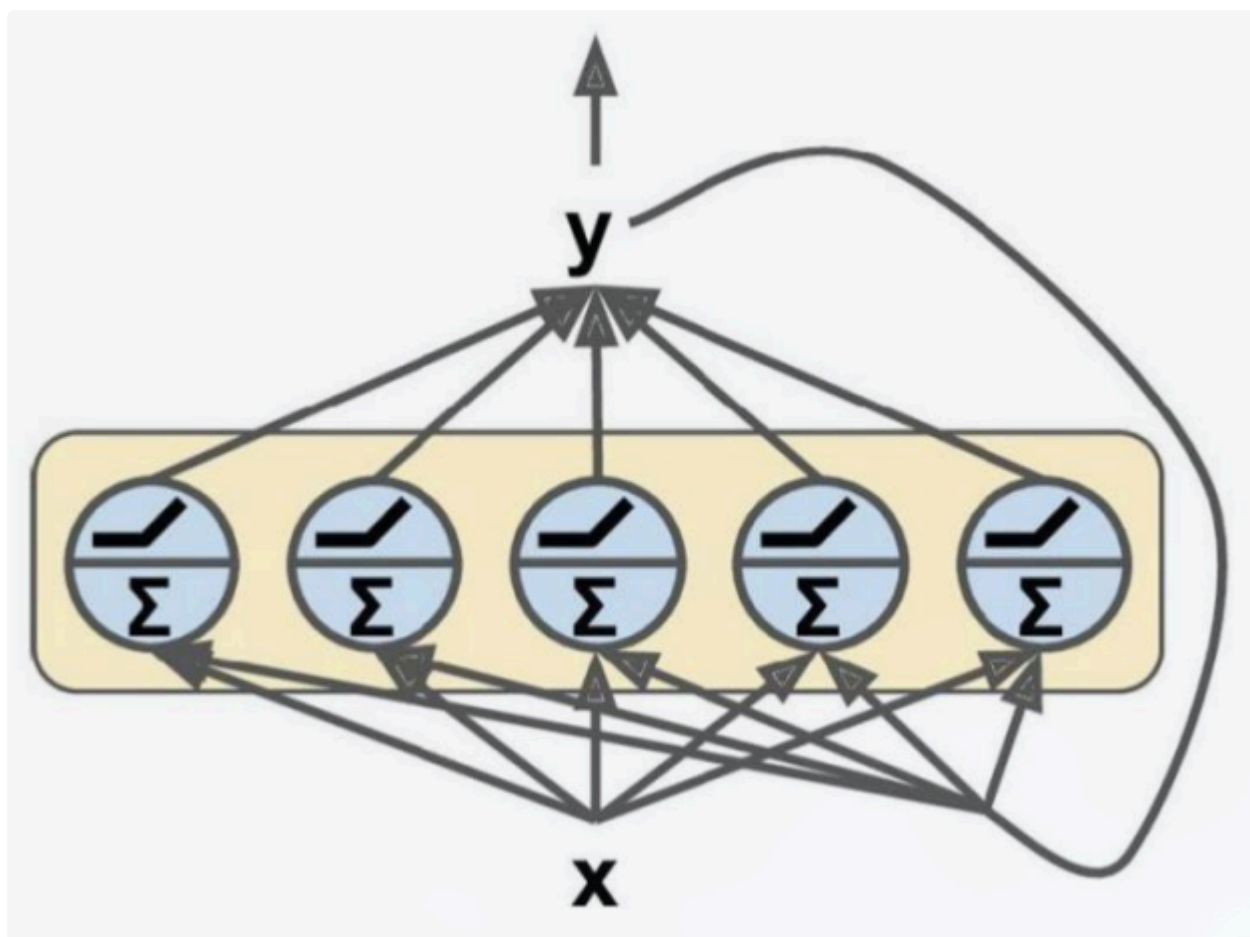
RECURRENT NEURAL NETWORKS

To allow neural networks to **process sequences**, we can introduce **connections** between **neurons of the same layer** and also between **non-adjacent layers** (recall that MLPs only featured connections between *adjacent layers*). The most widespread models (e.g. LSTM and GRU) employ *connections towards the same layer*.

WITHIN-LAYER CONNECTIONS

These connections are useful because they allow the layer's **input** to be a **combination** of a *"new"* input and the **previous output**.

To better understand this concept, consider a simple **3-layer network** (input-hidden-output) where the *middle layer* features these *within-level connections*, just like the one shown below:



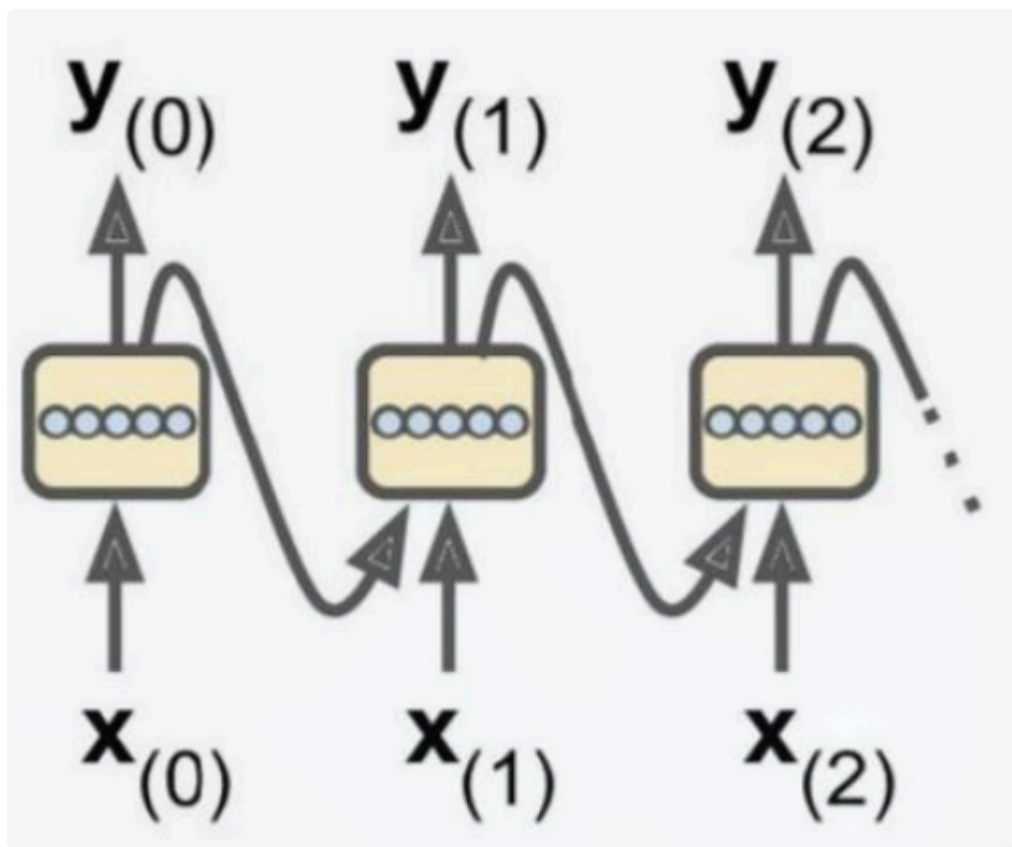
Notice that the **input** of **each neuron** is a **combination** of the **new input** **x** and the **previous output** **y**.

Now, one may wonder *how* and *why* this is **useful** for processing **sequences**.

To understand this, imagine that we want to process a **sequence** **x** that is made up of **"individual components"** (*terms*) $x_{(0)}, x_{(1)} \dots x_{(n)}$; that is:

$$\mathbf{x} = \mathbf{x}_{(0)}, \mathbf{x}_{(1)}, \dots, \mathbf{x}_{(n)}$$

If we **feed** these *terms* to the network **one at a time**, when we reach the i -th component $\mathbf{x}_{(i)}$, the network will **also** take as **input** $\mathbf{y}_{(i-1)}$, that is, the **result** associated with the *layer's output* for the **previous component** $\mathbf{x}_{(i-1)}$ (which itself depends on $\mathbf{x}_{(i-2)}$, and so on), just as shown in the figure below:



In simple terms, these connections allow us to **feed the sequence** to the network **one piece at a time**.

🔍 OBSERVATION: INPUT LENGTH

Notice that this implies that the **input sequence** can be of **any length**: all we have to do is **split it** into individual **components** $\mathbf{x}_{(i)}$ that *match the input layer dimension*.

Since we have stated that RNNs **process sequences**, but we haven't specified **what kind of sequences**, we clarify it now, before diving into more details.

✍️ RNN INPUTS

We mentioned **text** (sentences) as an example of sequences that can be processed with RNNs. Indeed, text is perhaps the most common kind of input for these networks, but, clearly, other sequences can also be fed to a RNN. Particularly, **videos** are **sequences of images** and thus, we process them by **feeding one frame at a time**.

MEMORY OF STATE

Other than allowing us to *process a sequence by breaking it apart* into smaller components, the added connections **also** allow our network to **maintain a memory of its state $\mathbf{h}_{(t)}$ for each instant** in time t .

In fact, **maintaining this information is only possible thanks to within-level and backward connections**: in a traditional *feed-forward* MLP, *information only flows forward* and thus *can't be "stored"* for future computations.

With this in mind, we define what a **cell** is:

📖 CELL

A cell is a **part of a recurring neural network** (consisting of a **predetermined number of neurons**) that **preserves an internal state (memory) $\mathbf{h}_t$** for each instant in time.

The cell is essentially a **layer with connections to its own neurons**.

In the simplified model we've seen above (*input-hidden-output*), the network only consists of an **input layer**, a **cell**, and an **output layer**.

Formally, the **state is computed** as a **function of the new input $\mathbf{x}_{(t)}$ and the previous state $\mathbf{h}_{(t-1)}$** : $\mathbf{h}_{(t)} = f(\mathbf{h}_{(t-1)}, \mathbf{x}_{(t)})$.

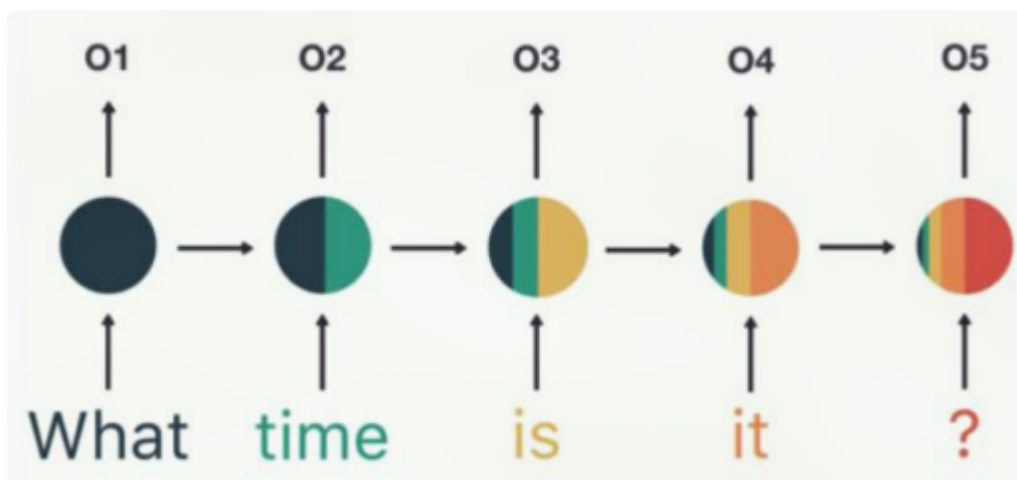
Since we are still in a neural networks setting, this simply means:

$$\mathbf{h}_{(t)} = \phi \left(\mathbf{x}_{(t)}^\top \cdot \mathbf{W}_x + \mathbf{h}_{(t-1)}^\top \cdot \mathbf{W}_h + b \right)$$

Where $\mathbf{W}_x$ are the **weights** associated with the **input**, $\mathbf{W}_h$ the **weights** associated with the **state**, b is the **bias** and ϕ is the **activation function**.

Because the network **maintains a state that carries information from the previous inputs**, it can **capture dependencies across the entire sequence**.

To understand this, consider the following example:



💡 THE STATE MAINTAINS INFORMATION

When we feed the **first term** of the sequence, the **state entirely depends on it**. Then, every time we **feed another term**, the **new state** will **add the new information** to the **previous state**.

At the **end**, the **state** of the cell **contains information** about the **entire sequence**.

Now, it is important we clarify a few details about the *cell's output*, and the *overall network's output*.

✍ CELL OUTPUT AND NETWORK OUTPUT

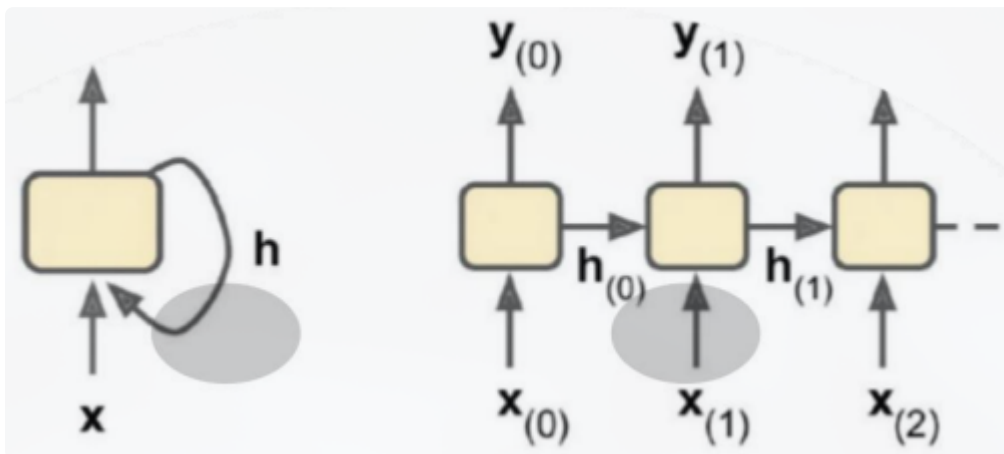
The **output of a cell** is simply **its own state**; that is, $y_{(t)} = h_{(t)}$.

However, the **overall output** of the **network** is obtained by **passing this state** to an **output layer** that performs the **final transformation**.

Essentially, this means that the **cell accumulates information** about the input sequence, so that at the **end** the **output layer** can **look at the entire**, "**condensed**" information to compute the desired output.

UNFOLDING IN TIME

Let us now **unfold our network in time** and make a few considerations:



In the image above, the network is displayed on the left, with its *same-layer connections*. On the right, we see **how** the same network **operates** on the **sequence** at **each instant in time**.

We notice that this "**unfolded in time**" representation of the network **looks a lot like** a standard **deep neural network**: it has a clear *forward path of information* (in the time direction), where each layer takes an **input**, **applies weights and biases**, an **activation function** and then **passes it to the next layer**.

This is indeed true, but we need to **pay attention** to a **crucial detail**: each layer **keeps applying** the **same weights** at **every time step**.

This is because the **underlying network** is **always the same**, we are simply "re-applying" it to each new term in the sequence.

💡 WEIGHT SHARING

An **unfolded RNN** on n **steps** is **equivalent** to a **DNN** with n **layers**, where all layers **share** the exact **same weights** (and biases).

Because of this, we say that **weights are shared over time**.

Now, if we wanted to be *really* pedantic, we could point out that there is actually *one more difference* with a *standard DNN*: that is, in the unfolded RNN, each layer is *also receiving its own input* and not just the output of the previous level.

In reality, we *do not care about this detail*. In fact, we only presented this **equivalence** to ease the understanding of the **exploding/vanishing gradient problem** for *recurrent neural networks* (recall that we had already encountered this in > [1.2 - DEEP NEURAL NETWORKS > EXPLODING/VANISHING GRADIENT PROBLEM](#)).

Clearly, in order to train such a network, we need to use **examples** of **varying lengths**. When these examples are **particularly long**, we encounter two problems:

✍ EXPLODING/VANISHING GRADIENT

1. The **longer the example**, the **more "layers"** (as in the analogy with DNNs) we are **going through**: deeper networks **more easily suffer** the exploding/vanishing gradient problem.
2. Because we are **always applying** the **same weight matrix**, we need to be **particularly careful** with the **values** that the weights take: if these are **too low**, repeatedly applying them leads to a **vanishing gradient**; contrarily, if these are **too high**, their repeated application will result in an **exploding gradient**.

Exploding gradient can be solved with **gradient clipping techniques**.

In its most simple form, *gradient clipping* consists in **rescaling down** the **gradient** if it **gets too large**.

📖 GRADIENT CLIPPING

More precisely, if at any point we have:

$$||\nabla_{\theta}\mathcal{L}|| \geq c$$

We scale down the gradient to:

$$\nabla_{\theta}\mathcal{L} = c \frac{\nabla_{\theta}\mathcal{L}}{\|\nabla_{\theta}\mathcal{L}\|}$$

Notice that $\frac{\nabla_{\theta}\mathcal{L}}{\|\nabla_{\theta}\mathcal{L}\|}$ is a **unit vector** with the **same direction** of the **original gradient**; thus, after clipping, the gradient will simply be **rescaled** to have **norm equal to c**.

✍ ADDRESSING VANISHING GRADIENT

Solving the **vanishing gradient** problem, on the other hand, is **much harder**. This is done in several different ways:

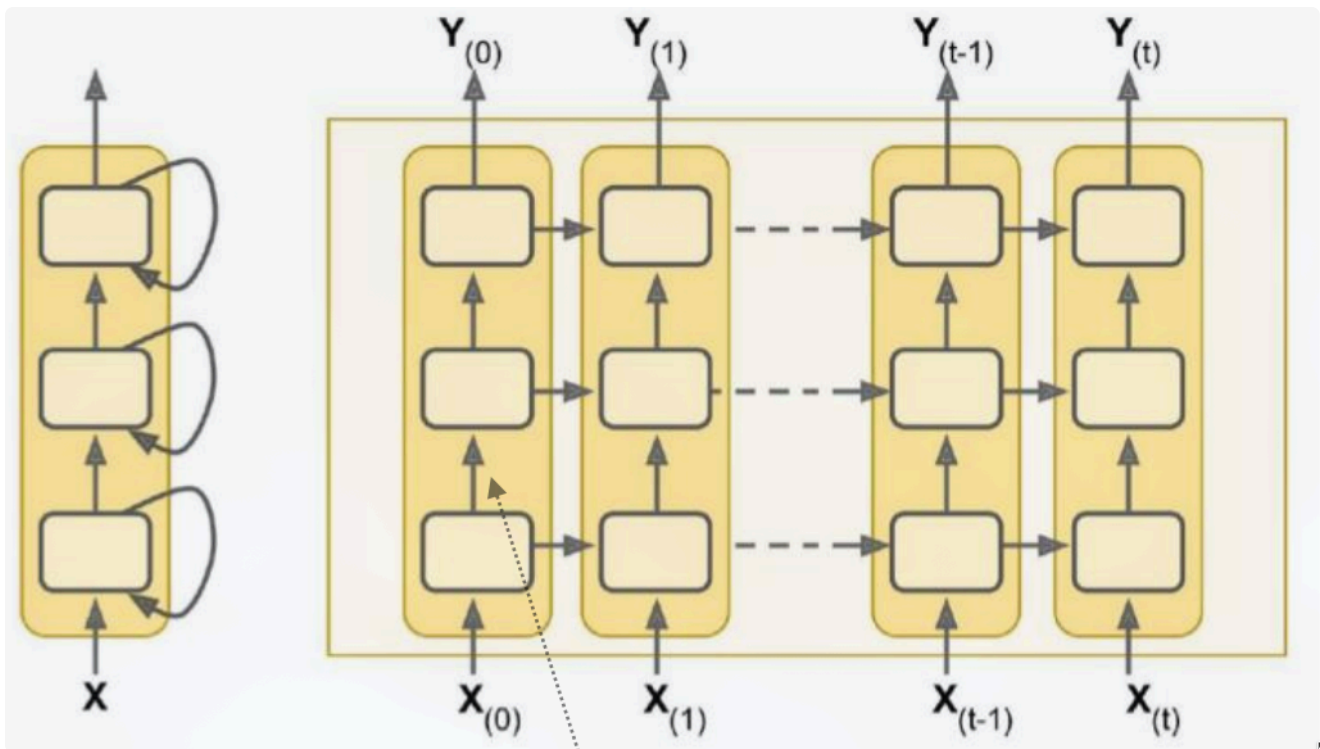
1. **Architectural solutions:** by introducing some changes to the network architecture, we can *"force" gradients to flow more smoothly* (see > [LONG SHORT-TERM MEMORY](#) below).
2. **Activation functions:** these also contribute to the gradient; thus cleverly choosing activation functions that are *well-suited for deep networks* can help mitigate the problem.
3. **Orthogonal initialisation:** if we initialise the *weight matrix* $\mathbf{W}_h$ to be an *orthogonal matrix* (whose eigenvalues have unit magnitude), the *gradient norm* can be preserved. Notice that this is a *weak solution*, as weights are *not the only values* that *affect the gradient* (as already pointed out, the activation function also influences it).

DEEP RNNs

In the above paragraphs, we only discussed **single-layer RNNs**. Indeed, there exist **more complicated RNN architectures**, such as **LSTM** and **GRU**.

Before diving into these, it is worth introducing **Deep RNNs**, which can be obtained by **stacking** the **base cell architecture** we've seen above.

The figure below shows an RNN consisting of three stacked layers (network architecture on the left, unfolding in time on the right):



In general, instead of a *single* recurrent layer, we have L layer stacked on top of each other.

This means that:

🔍 OBSERVATION

- **INFORMATION FLOW:** At each time-step t , the **hidden state of layer l ($h_{(t)}^l$)** is **passed up** as input to the **next layer $l + 1$ ($x_{(t)}^{l+1}$)**.
- **RECURRENCE:** Each **layer l** also **maintains its own separate state**, passing $h_{(t)}^l$ to $h_{(t+1)}^l$.

Just like in CNNs, where depth allowed for **hierarchical representations**, **stacking** more layers allows **lower levels** to **learn simple patterns** (e.g. word syntax) and **higher levels** to **combine them** into more **complex, abstract concepts** (e.g. general semantics).

📝 NOTE

When **training** a Deep RNN we **only** provide the **desired inputs** and **outputs** for the **last level**; that is, we **only compute loss** for the **very top layer**.

LONG SHORT-TERM MEMORY

The **basic RNN architecture** presented above has a **problem**: the cells have **difficulty remembering** inputs from "**distant**" steps.

✎ INEFFICIENT MEMORY

This tendency to **forget the first terms** (which are "further" from the final time-steps) implies **poor performance** when dealing with **long sequences**.

This problem is *similar to that of the vanishing gradient* due to the fact that the **state** is **computed** with **repeated multiplications** with the weight matrix W_h .

To address this issue (as well as **mitigating** the **vanishing gradient** problem, since they share similar causes), we need to **modify** the **architecture of the base cell**.

One possible solution is given by **Long Short-Term Memory (LSTM)** networks.

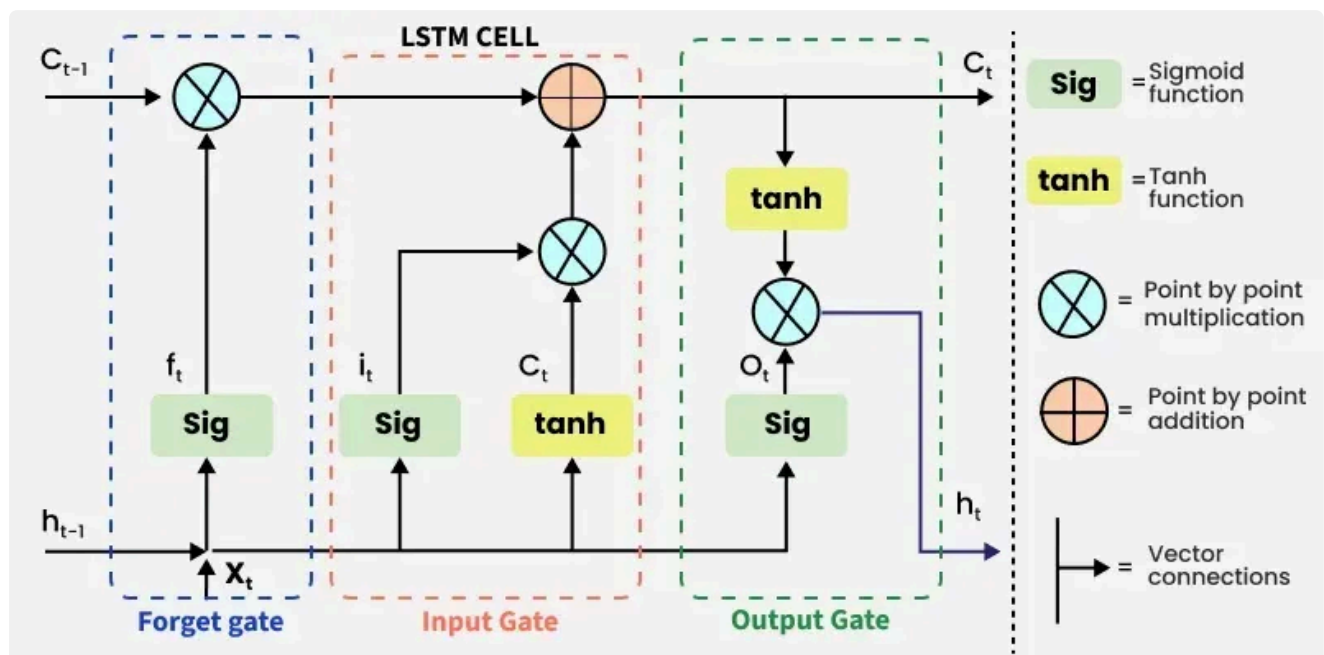
As the name suggests, they aim to **accompany** the **short-term memory** of RNNs with a **long term memory** that can **last over thousands of steps**.

✎ ARCHITECTURE CHANGE

The **main intuition** behind the LSTM architecture is to create an **additional module** in a neural network that learns **when to remember** and **when to forget** pieces of information.

More specifically, a **LSTM cell** is composed of **three gates** (*input, output, and forget*), which **regulate the flow of information** through the cell.

The image below displays the **organisation** of these **gates** inside the cell:



Notice that the cell now **maintains two states**: a **cell state** c_t , and a **hidden state** h_t .

To understand *what role they play* in the network, let us give a detailed explanation of *how the gates work and cooperate* (notice that we use $\odot$ to denote *element-wise multiplication*):

📋 LSTM CELL

1. FORGET GATE

This gate decides **what information to keep** (and *what to discard*) from the **previous cell state** $\mathbf{c}_{t-1}$.

Mathematically, it is computed as:

$$\mathbf{f}_t = \sigma(U_f \mathbf{x}_t + W_f \mathbf{h}_{t-1} + \mathbf{b}_f)$$

2. INPUT GATE

This gate generates a **processed input** $\mathbf{i}_t$ and a **candidate state** $\bar{\mathbf{c}}_t$.

These are computed as:

$$\mathbf{i}_t = \sigma(U_i \mathbf{x}_t + W_i \mathbf{h}_{t-1} + \mathbf{b}_i)$$

$$\bar{\mathbf{c}}_t = \tanh(U_c \mathbf{x}_t + W_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

The **gate's output** is then computed as $\mathbf{i}_t \odot \bar{\mathbf{c}}_t$.

3. OUTPUT GATE

Finally, this gate **updates the states** of the cell.

Particularly, it computes the **new cell state** as:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \bar{\mathbf{c}}_t$$

Then, it computes:

$$\mathbf{o}_t = \sigma(U_o \mathbf{x}_t + W_o \mathbf{h}_{t-1} + \mathbf{b}_o)$$

And finally it **updates the hidden state** as:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Where $\mathbf{x}_t \in \mathbb{R}^d$ (with d denoting the *number of input features*), all other vectors (denoted in bold) are in $\mathbb{R}^h$ (where h is the *number of hidden units*), $U_{\square} \in \mathbb{R}^{h \times h}$, and $W_{\square} \in \mathbb{R}^{h \times d}$.

Now, let us make some further **observations** about **each computed value***:

🔗 OBSERVATIONS

1. The **forget gate output** is $\mathbf{f}_t \in (0, 1)^h$: each component acts as a **selector** of **how much information to retain** from the **previous cell state** when we compute $\mathbf{f}_t \odot \mathbf{c}_{t-1}$.
2. The **candidate state** $\bar{\mathbf{c}}_t \in (-1, 1)^h$ **proposes** both **additions** (positive values) and **subtractions** (negative values) to the **cell state**.

3. The **processed input** $\mathbf{i}_t \in (0, 1)^h$ decides the **importance** of each **proposed addition/subtraction** when we compute $\mathbf{i}_t \odot \bar{\mathbf{c}}_t$.
4. The **new cell state** takes into consideration both **old retained memory** and **new information**.
5. Finally, the **hidden state** $\mathbf{h}_t \in (-1, 1)^h$ (which is also the **output of the cell**) depends on both an **evaluation** of the **new input** and **previous state**, as well as the **current cell state**.

Maintaining two separate states may look **excessively complicated**, especially given that the **output** of all gates is a **function** of both the **new input** at time t and the **previous state** at time $t - 1$.

We may thus ask ourselves:

*If all the gates involved depend on the same inputs, do we really need to keep **two** different states?*

To answer this question, it is important to look at one key **characteristic** that distinguishes the two; that is, $\mathbf{c}_t$ is **unbounded**.

We explain below what this means:

SHORT TERM MEMORY AND LONG TERM MEMORY

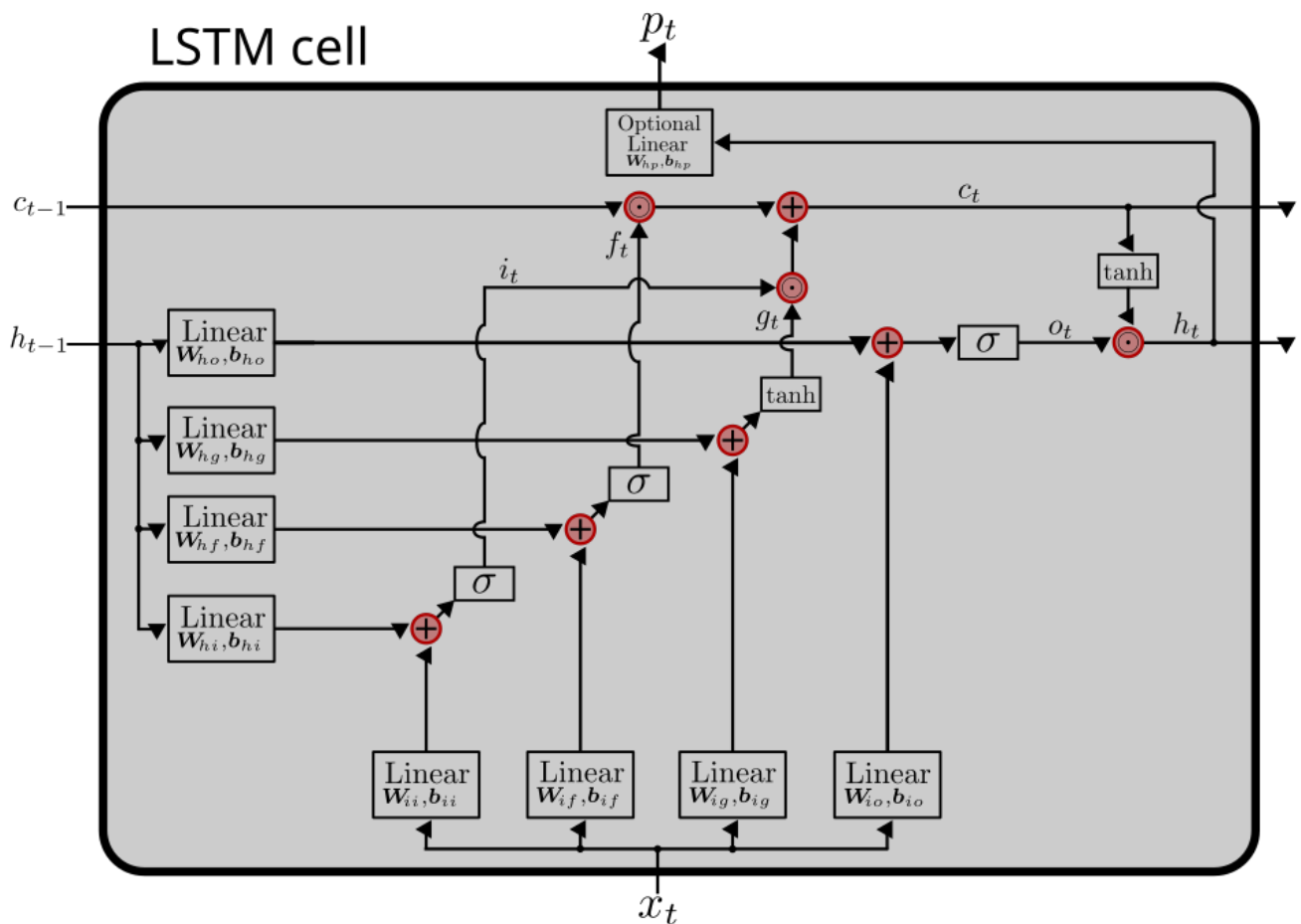
- The **hidden state** is **bounded**: it takes a "**squashed**" version of the **cell state** and **applies the output gate** to it, which we can think of as a "**relevance filter**". In practice, $\mathbf{h}_t$ decides what parts of $\mathbf{c}_t$ contain **information relevant to the current time-step**.

We can thus think of $\mathbf{h}_t$ as a **short term memory**.

- Conversely, $\mathbf{c}_t$ is **unbounded**: it **keeps accumulating information** over the course of a **larger number** of time-steps.

We can thus think of it as a **long term memory** (intuitively, a **global context** that maintains information about the whole sequence).

Finally, for the sake of completeness, a **more detailed** depiction of the **flow of information** through a LSTM cell is shown below:



Notice that h_t is also the **cell's output**: it can be fed to another cell if the network features a **stacked architecture**, or it can be **passed through** another **weight layer** to obtain the **final output**.

✍ FINAL NOTES

1. The **addition of gates** and, particularly, **addition operations** helps **mitigating the vanishing gradient problem**.
2. The **states h_t and c_t** can be **initialised by setting them to zero**, at **random**, or even through more **clever methods**. Particularly, it's been shown that **non-zero initialisation** can help **speed up training**.

Bi-LSTM

An interesting way of using LSTMs is employing **two of them** to **process a sequence in both directions** (front to back, and back to front).

Such architecture is known as **bi-directional LSTM (Bi-LSTM)** and is particularly **well suited** for applications such as:

- **Speech recognition**
- **Text classification**
- **Forecasting models**

To understand why **processing in both directions** might be **helpful**, consider the following example:

≡ EXAMPLE

Imagine we have **two sentences**:

1. *The boys go to*
2. *Boys come out of school.*

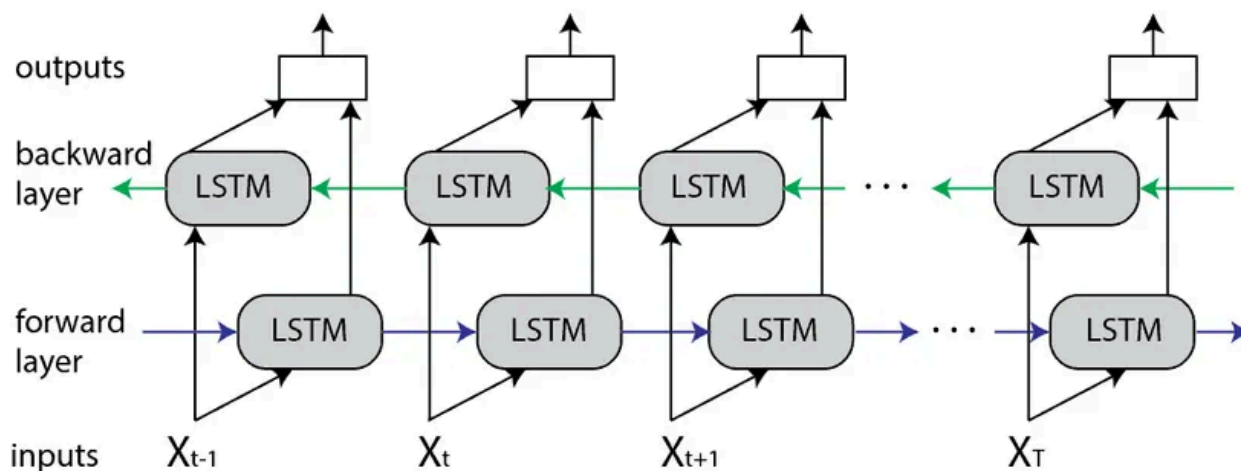
And we want to **predict** what should appear where the **blank spaces** are.

If we **feed both sentences** to the **two networks** in **opposite order** (that is, network 1 processes the first sentence from "The" to the blank space, while network 2 processes the second sentence back to front), **by the time** the first network **reaches the blank space**, the second network has **already seen "out of school"**, which may help the former **correctly predict** that the blank space may be "school".

💡 INTUITION

If we **combine the past context** (from the **forward pass**) with the **future context** (from the **backward pass**), we can produce **much better predictions**.

The **flow of information** through the two networks looks as follows:



As we can see from the figure, the **output** for a **term x_t** in the **sequence** depends on the **hidden states** of **both networks**.

💡 OBSERVATION

This means that the **output** for x_t becomes **available only after** $\max\{t, T - t + 1\}$ **steps**: that is, until **both networks** have **processed the term**. Consequently, the **output** for the **whole sequence** is only **available after T steps**.

This highlights one **key limitation** of **Bi-LSTMs**: they **can't be used for real-time predictions**, because *no output can be finalised until both networks have processed the whole sequence*.

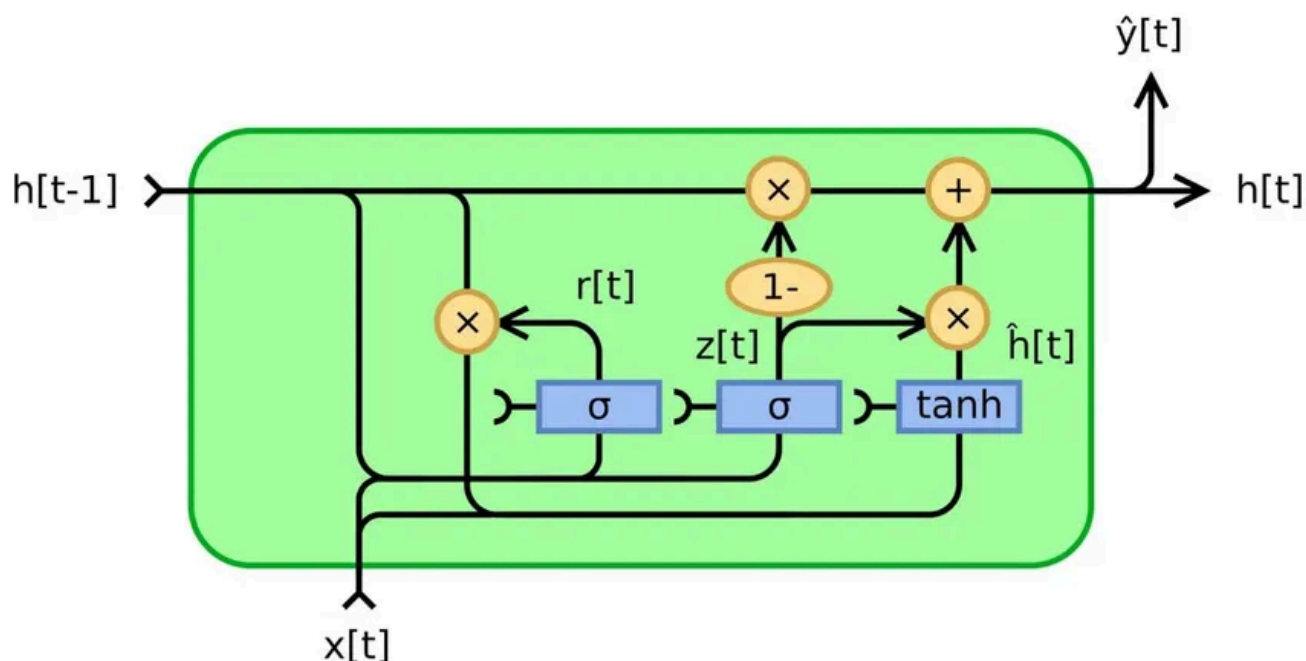
Furthermore, there exist **different possible ways to combine the outputs**; the following are the most commonly used: **concatenation**, **multiplication**, **addition**, and **average**.

✎ CLARIFICATION

The **two networks** aren't connected (they are **independent**): we **simply combine their outputs** to produce the **final prediction**.

GRU

Gated Recurrent Units (GRUs) are a **simplified version of LSTM cells**, which try to **maintain their advantages while reducing parameter count and complexity**.



💡 SIMPLIFYING LSTMs

As we can see from the figure above, the main simplification comes from the fact that we only maintain **one single state of memory h_t** .

The basic components of a GRU cell are a **reset gate r_t** , which determines **how much old information to forget**, and an **update gate z_t** which determines **how to compute the new state** by weighting the contributions of the **old state** and a **new candidate state**.

These gates are defined as follows:

$$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z)$$

$$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r)$$

We then define a **candidate activation vector** as:

$$\hat{\mathbf{h}}_t = \tanh(W_h \mathbf{x}_t + U_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

Finally, the **state** is **updated** as follows:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \hat{\mathbf{h}}_t$$

TEMPORAL CONVOLUTIONAL NETWORKS

Temporal Convolutional Networks (TCNs) are an alternative way to process sequences.

The main idea behind them is to use **1D convolutions over the length** of the sequence.

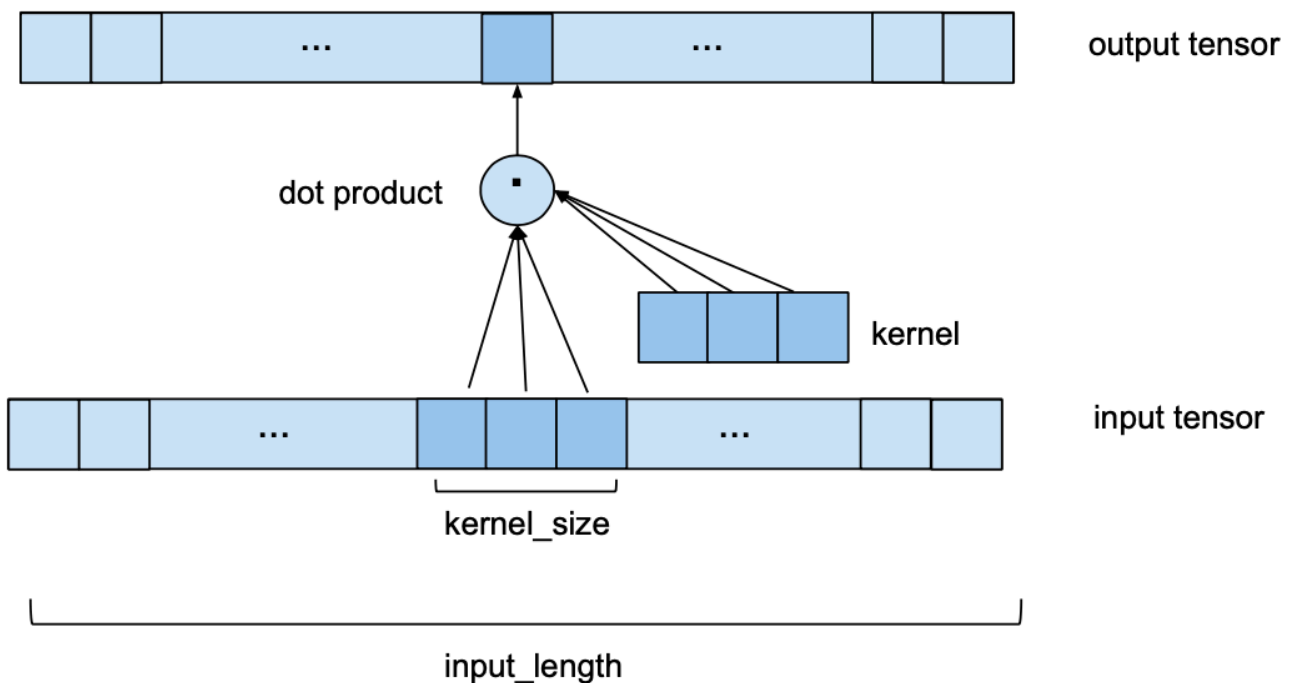
The **1D convolutional layers** receive as **input** a tensor of **shape**:

$$\text{batch size} \times \text{input length} \times \text{input channels}$$

And **output** a **tensor** of **shape**:

$$\text{batch size} \times \text{input length} \times \text{output channels}$$

To see how one single layer processes its input, let us look at **one element of the batch** (after all, the same process is *simply applied to all elements in the batch*) and, for simplicity, let us consider the case where both **input** and **output** only feature **one channel**:

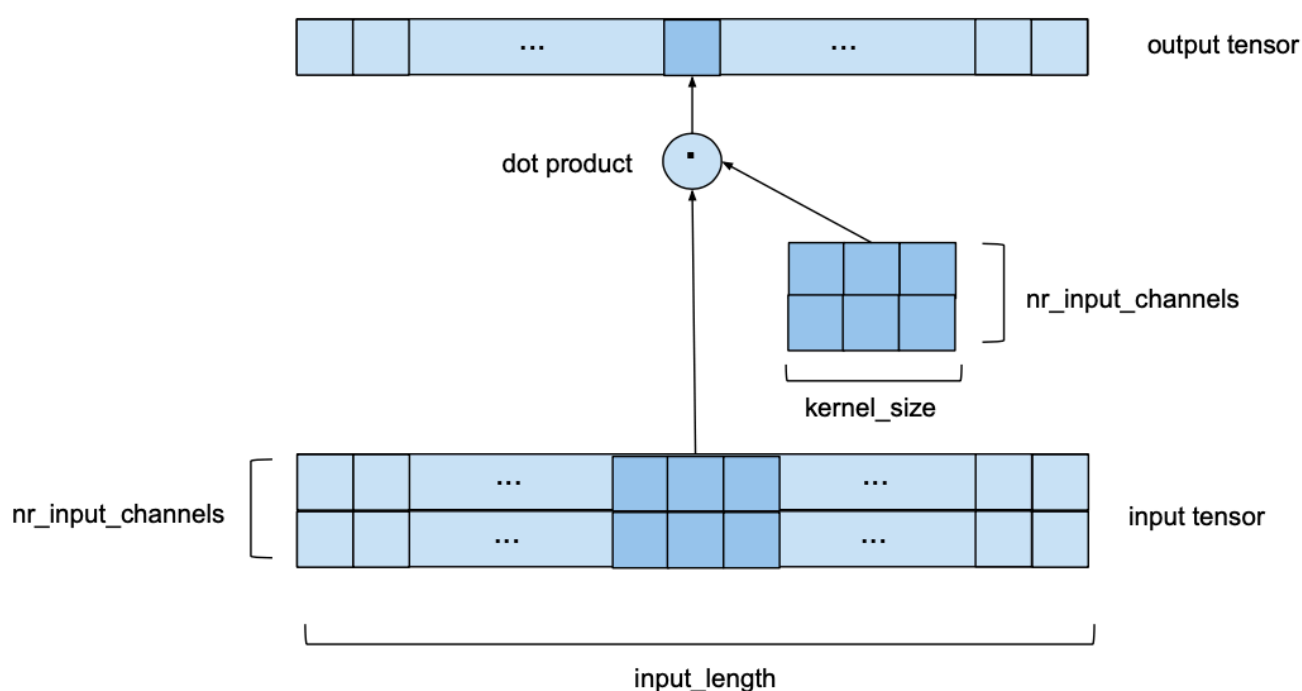


We can see that to compute one element of the output, we look at a **series of consecutive elements** of length **kernel_size** of the input.

To obtain the output, we simply take the **dot product** of the sub-sequence of the input and a **kernel vector of learned weights** of the same length.

To get the **next element** of the output, the same procedure is applied, by **shifting the kernel** to the **right** by one element (here, we use unit stride; different stride values can also be employed).

If the **input** has **multiple input channels**, we simply need to **adjust the dimensions** of our **convolution kernel**, as shown below:



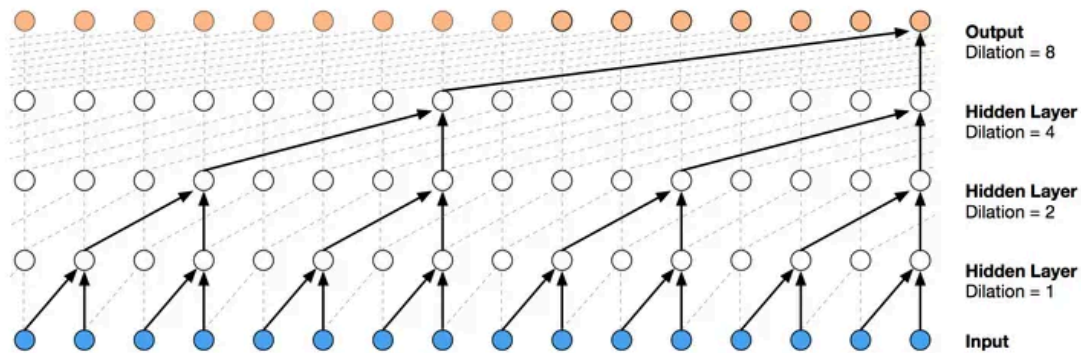
NOTE

The **number of output channels** can also be different from one. Just like we did with CNNs, we simply need to **apply as many filters** as the **desired number of output channels**.

DILATION FACTOR

To **build up context** from **previous time-steps**, multiple **convolutional layers** are **stacked** on top of each other.

Furthermore, to achieve **larger receptive field sizes**, at each successive layer we **increase** the **dilation factor** of the kernel (that is, the kernel is applied to **distant elements** instead of adjacent ones) **exponentially**, as shown in the image below:



📖 DILATION FACTOR AND RECEPTIVE FIELD

At layer $l = 0, 1, \dots$ the dilation factor is given by:

$$d_l = 2^{l-1}$$

Assuming we use **kernels of size k** and **unit stride**, the **receptive field** is given by:

$$R_l = (k - 1)d_l + 1 = (k - 1)2^{l-1} + 1$$

It is also worth giving a reason for the *exponential* increase of the dilation factor:

💡 OBSERVATION: EXPONENTIAL DILATION

Given a **fixed dilation factor**, the **number of layers** required for **full history coverage** is **linear in the length of the input tensor**, which will result in networks that become **very deep very fast**.

By **increasing the dilation factor exponentially**, we can **greatly reduce the required depth** to process a whole sequence.

CAUSAL CONVOLUTIONS, PADDING AND NETWORK ARCHITECTURE

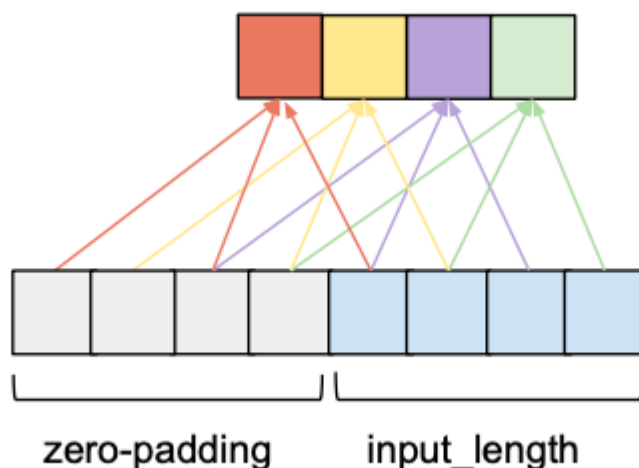
Now, ideally we want the **i -th element** of the convolution **output** to **only depend** on the elements of the input sequence at **previous indices**; that is, $i \in [0, i]$.

This is simply the **idea of causality**, and can be achieved with **causal convolutions**:

📖 CAUSAL CONVOLUTIONS

The activation outputs computed for a **particular time-step** do **not depend** on the outputs of **future steps**.

To actually implement this, we need to apply **zero-padding** (that is, add "artificial" zeroes) to the **left of the sequence** so that **kernels** at **different layers** can be **applied correctly**:



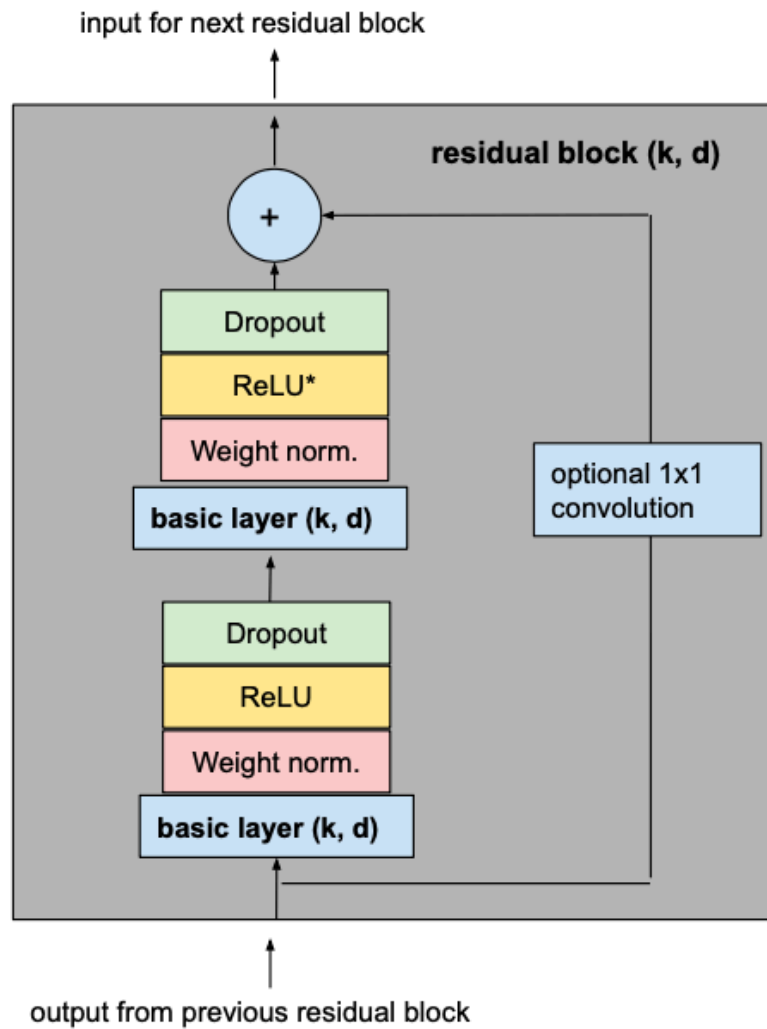
Now that we know all the *individual pieces* at play in a **TCN**, we can finally look at its **overall architecture**.

To start, **TCNs** use the above discussed **causal convolutional layers** inside **residual blocks** (recall that these have been discussed in > [2.0 - CONVOLUTIONAL NEURAL NETWORKS > RESNET](#)).

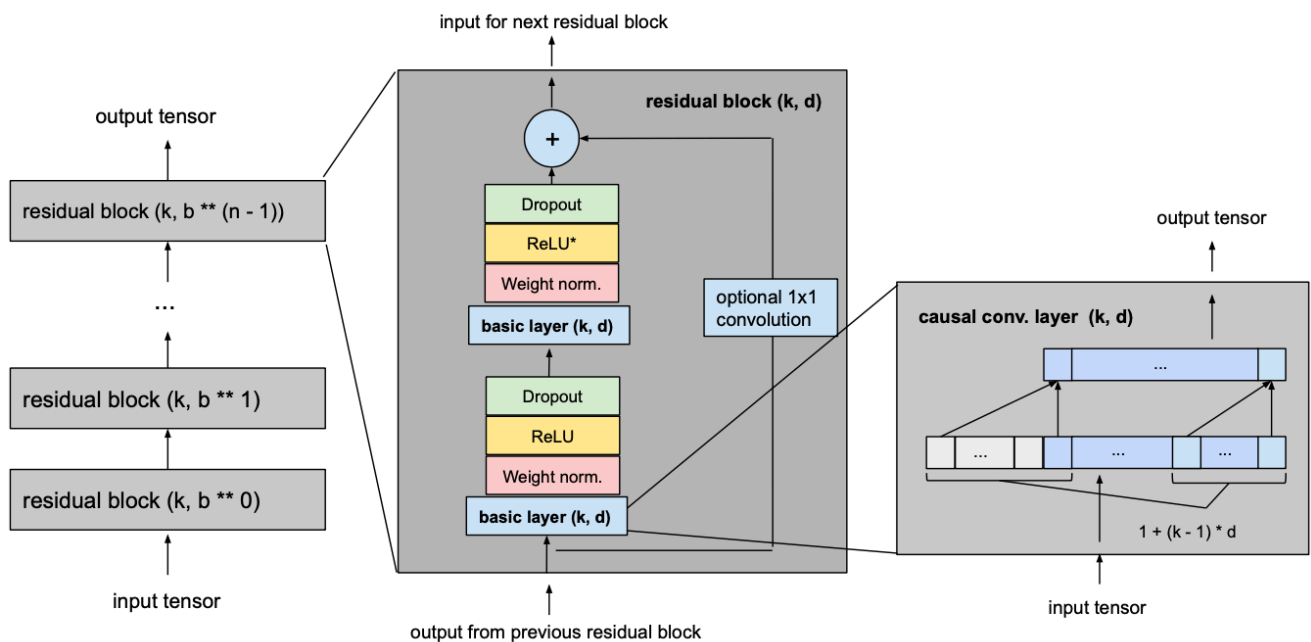
Furthermore, **each block** also features:

- **Weight normalisation** layers.
- **ReLU** activations.
- **Dropout** after every activation layer.

The resulting **residual block** is shown below:



Finally, these blocks are **stacked** together to obtain the following architecture:



TIME SERIES FORECASTING

Finally, we conclude the chapter with one of the **main applications** of TCNs: **time series forecasting**.

Intuitively, a **time series** is simply a **trend** that **changes over time**; think of **stock prices**, the **temperature** of an **environment**, the **energy consumption** of a system, and so on.

In all such cases, it may be useful to develop a model to **predict the future trend** of the time series: we call this task **forecasting**.

📖 TIME SERIES FORECASTING

Given a time series $f(t)$ as a **collection of samples** at **different time-steps** $t \leq T$, we are interested in **predicting the value** of f for **future time-steps** $t > T$.

📌 NOTE: INPUT CHANNELS

If $f(t) \in \mathbb{R}$ our input sequence only comprises a **single channel**.

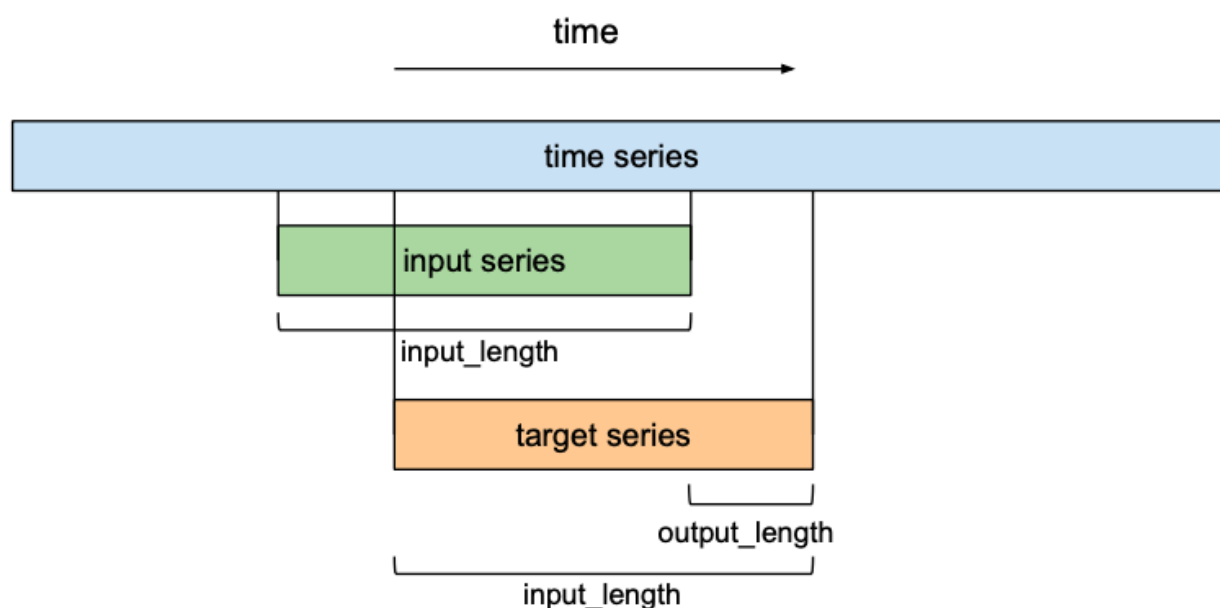
If $f(t) \in \mathbb{R}^n$, our input sequence features **n input channels**.

To **train** our TCN to do forecasting, the **training set** will consist of pairs of the form (input sequence, target sequence), where both input and target are equally-sized **sub-sequences** of the given time series.

A **target** series will be a series that is **shifted forward** in relation to its **respective input series** by a certain number **output_length**.

This means that a **target** series of length **input_length** contains the **last (input_length – output_length)** elements of its **respective input** sequence as **first elements**, and the **output_length** elements that **come after** the last entry of the input series as its **final elements**.

This is illustrated in the figure below:



COMPARISON OF TCNs AND LSTMs

We have seen that **LSTMs** process sequences **one term at a time**, while **TCNs** process the **whole sequence in one go**.

As a consequence:

💡 LSTMs VS TCNs - PARALLELISM

Because **TCNs** can process the entire sequence simultaneously, they allow **parallelisation on GPUs**, which translates to **faster computations over the entire sequence**.

On the other hand, the ability of **LSTMs** to **process sequences in bits** makes them inherently better at **real-time evaluation** of data (e.g. streaming services).

Furthermore, by looking at the whole sequence at once, **TCNs** often **achieve better performance** when **compared to LSTMs** with a comparable number of parameters. This is due to the fact that **kernel size**, **stride** and **dilation** can be tweaked to achieve a **precise control** over the **receptive field**, thus making **ad hoc configurations** possible. In any case, it is important to remark that this also implies that **TCNs** have a **bigger memory footprint** due to the fact that the **whole sequence** needs to be **stored in memory**.

📋 RECAP: TCN VS LSTM

We usually choose to use a **TCN** when:

- The **entire sequence** is **available at once**.
- We need to **capture long range dependencies**.
- The **primary constraint** is **inference speed**.

Contrarily, we choose a **LSTM** network when:

- We're dealing with **real-time processing**.
- We are **memory-constrained**.
- Sequences are **very long**.

GRAPH CONVOLUTIONAL NETWORKS

Graph Convolutional Networks (GCNs) were first introduced in "*Spectral Networks and Deep Locally Connected Networks on Graphs*" by Bruna et al. in 2014 as a method for **applying neural networks to graph-structured data**.

One may wonder **why** we would want to process graphs with neural networks. The answer lies in the following fact:

WHY GRAPHS

Graphs provide a way to **bring structure** to otherwise **unstructured data** by explicitly **modelling relationships** between **entities**.

After all, many **problems** that received attention at the **time** were (and still are) related to data that can be **intuitively represented** with **graphs**; namely, **chemical compounds** (particularly, *genes* and *drugs*) and **text** (natural language processing has been one of the main applications for machine learning).

In general, many *graph-related* problems often boil down to one of the following:

- **Graph classification** - Given a graph, predict which class it belongs to.
- **Node classification** - Given a graph with incomplete node labels, predict the classes of the remaining labels.
- **Link prediction** - Given a graph with missing edges, predict where these might be.
- **Graph clustering** - Given a graph, partition its nodes into clusters based on its edge structure.
- **Graph embedding** - Given a graph, map it into a vector while preserving relevant information.
- **Graph generation** - Learn a distribution of graphs with particular features and sample it to generate similar graphs.

The main idea behind GCNs is to implement a **form of message passing** between **nodes**:

MESSAGE PASSING

If we can **define** a process that **passes information between nodes** of a graph, we can use it to build a **network** that **learns to extract features** from **each node** that **depend on its connections** to the **other nodes** in the graph.

Thus, the following paragraphs will first **define** the **required mathematical operations**, and then show how we can employ them to build a **network capable of processing graph data**.

GRAPHS REVISION

Because GCNs operate on graphs, it is worth going through a quick revision on graphs, terminology and operations on graphs.

We begin with the definition of graph:

GRAPH

A graph G is a triplet

$$G = (V, E, W)$$

Where:

- V are **vertices** (or **nodes**), which are defined as a set of N **numerical labels** $V = (1, \dots, N)$.
- E are **edges**, defined as **ordered pairs of nodes** (i, j) with $i, j \in V$.
- W are **weights**: $w_{ij} \in \mathbb{R}$ is the **strength of the influence** j has on i .

Now, recall that we can distinguish **directed** and **undirected** graphs:

📖 DIRECTED AND UNDIRECTED GRAPHS

In a **directed graph** we have $(i, j) \neq (j, i)$: that is, $\exists(i, j) \nRightarrow \exists(j, i)$.

Furthermore, even if both edges exist, their **weights may be different**.

Contrarily, in an **undirected graph** we make **no distinction** between (i, j) and (j, i) .

Moving on to **graph representation**, we can use **matrices** to represent graphs.

📖 ADJACENCY MATRIX

Given a graph G with N vertices, its **adjacency matrix** A is:

$$A \in \mathbb{R}^{N \times N} \quad A_{ij} = w_{ij}$$

Notice that, if the graph is **undirected** we have $A = A^\top$.

📖 DEGREE MATRIX

Given a graph G with N vertices, its **degree matrix** D is a **diagonal matrix** defined as:

$$D \in \mathbb{R}^{N \times N} \quad D_{ii} = d_i = \sum_{j \in \mathcal{N}(i)} w_{ij} \quad D_{ij} = 0 \quad \forall i \neq j$$

Where $\mathcal{N}(i)$ is the set of nodes that are connected to i (**neighbourhood**) and d_i is called the **degree of the node**.

Finally, from these two matrices we define the **Laplacian matrix** and the **normalised Laplacian matrix**:

📖 LAPLACIAN MATRIX

Given a graph G with *adjacency matrix* A and *degree matrix* D , we define its **Laplacian matrix** L as:

$$L = D - A$$

We also define its **normalised Laplacian matrix** $\bar{L}$ as:

$$\bar{L} = I - \bar{A}$$

Where:

$$\bar{A}_{ij} = \frac{w_{ij}}{\sqrt{d_i d_j}}$$

GRAPH SIGNALS AND GRAPH SHIFT OPERATORS

Now, before diving into how we can **generalise convolutions to graphs**, we first need to define what a **graph signal** and a **graph shift operator** are.

Recall from *classical signal processing* that:

| A *signal* is a function that assigns a value to points in space or time.

We can thus apply this idea to graphs and define a **graph signal** as follows:

📖 GRAPH SIGNAL

Given a graph G with N nodes, a **graph signal** on G is a **vector**

$$\mathbf{x} \in \mathbb{R}^N$$

that **assigns a value** $x_i \in \mathbb{R}$ to **each node** i .

Now recall (again, from classical signal processing) that a **shift operator** acts on a signal by **shifting it in space or time**.

In the context of graphs, we can thus simply define a **graph shift operator** as follows:

📖 GRAPH SHIFT OPERATOR (GSO)

Given a graph G , a **graph shift operator** for G is a **matrix** S that:

1. **Encodes the structure** of G .
2. When applied to a **graph signal** $\mathbf{x}$ on G , it **produces a new signal** $\mathbf{y} = S\mathbf{x}$.

Notice that the definition is quite vague, but this is fine because, for your purpose, we only need to know that S **must capture the structure of the graph**.

This may be easier to understand if we consider for example $S = A$; we thus have:

$$\mathbf{y} = A\mathbf{x} \implies y_i = \sum_j w_{ij}x_j$$

🔍 OBSERVATION

Because A carries information about the **relationships between nodes**, the application of the **shift operator** $S = A$ results in a **signal** that **mixes** x_j with its **neighbouring values** x_i .

Contrarily, if we used some other matrix S that was **unrelated** to the **graph**, the obtained **result** would be **meaningless** in relation to G .

For our purposes, A , D , L and $\bar{L}$ are all **valid GSOs**.

Furthermore, notice that the **application of the operator** S **moves the signal** $\mathbf{x}$ **only one edge at a time**.

If we want to **propagate the signal further**, we need to **repeatedly apply** S .

With this in mind, we can define a **diffusion sequence** on the graph:

📖 DIFFUSION SEQUENCE

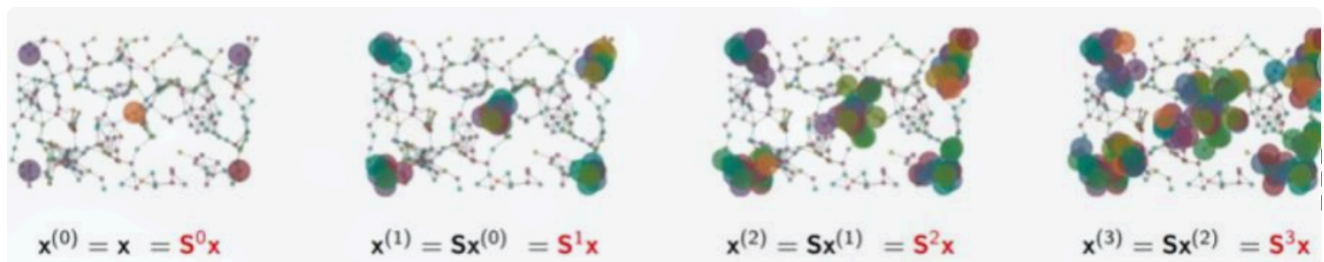
Given a **graph** G , a **GSO** S and a **graph signal** $\mathbf{x}$, we define a **diffusion sequence** over K steps as:

$$\mathbf{x}_{k+1} = S\mathbf{x}_k \quad \mathbf{x}_0 = \mathbf{x}, k \in [0, K - 1]$$

Or **equivalently**, in form of a *power sequence*:

$$\mathbf{x}_k = S^k \mathbf{x}$$

The figure below should help visualise the process:



🔍 OBSERVATION

Notice that the **information** about G **carried by the GSO** is what makes the **diffusion process possible**: that is, the **signal propagates** through the graph

according to the structure encoded by S .

GRAPH CONVOLUTIONS

Though the excursus about **graph signals** and **graph shift operators** may have seemed confusing, it was very much necessary; in fact, both concepts are important to define **graph convolutions**.

GRAPH CONVOLUTION

Given a **graph** G , represented by a **matrix** S , and given **filter coefficients** h_k , we define the **convolution filter** as the **polynomial** $H(S)$ given by:

$$H(S) = \sum_{k=0}^{\infty} h_k S^k = h_0 + h_1 S + h_2 S^2 + \dots$$

Then, the **graph convolution** of **filter** $H(S)$ with **coefficients** $\mathbf{h} = \{h_k\}_{k=0}^{\infty}$ on the **graph signal** $\mathbf{x}$ on G is the **signal** $\mathbf{y}$ given by:

$$\mathbf{y} = H(S)\mathbf{x} = \sum_{k=0}^{\infty} h_k S^k \mathbf{x}$$

As a *short hand notation*, this is also written as:

$$\mathbf{y} = \mathbf{h} \star_S \mathbf{x}$$

In practice, since **only a finite amount** K of **diffusion states** is actually **observed**, it follows that we can consider:

$$\begin{aligned} \mathbf{y} &= \mathbf{h} \star_S \mathbf{x} \\ &= h_0 S^0 \mathbf{x} + h_1 S^1 \mathbf{x} + h_2 S^2 \mathbf{x} + \dots + h_{K-1} S^{K-1} \mathbf{x} \\ &= \sum_{k=0}^{K-1} h_k S^k \mathbf{x} \end{aligned}$$

Now, one may (legitimately) wonder:

Why is the convolution filter a *polynomial* in S ?

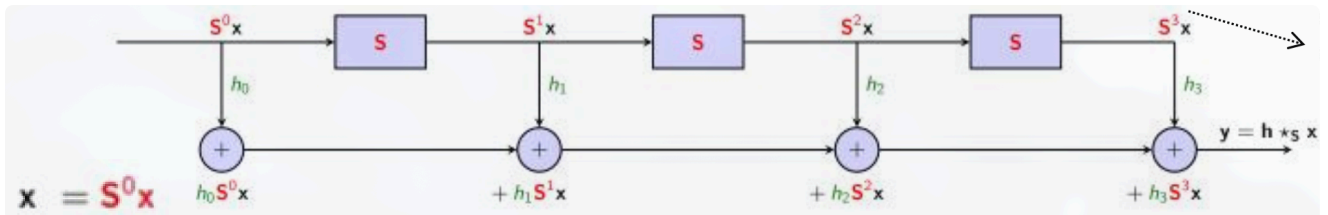
To understand why this makes sense, it is useful to think about what $S^k \mathbf{x}$ represents (hence why the diffusion process was necessary):

🔍 OBSERVATION: CONVOLUTION FILTER

Each $S^k \mathbf{x}$ term carries **information** from k "hops" away and is **weighted** by the respective **filter coefficient** h_k .

We can thus say that the convolution successively **aggregates information** from **local** to **global neighbourhoods** thanks to a linear combination of the **elements** of the **diffusion sequence**.

Notice that, in this optic, a graph convolution can be visualised in a **block diagram** as shown below:



Notice that we **iteratively apply** a **shift** of S , a **weighting** of $S^k \mathbf{x}$ with h_k , and a **summation** of each term.

GRAPH PERCEPTRON AND GCNs

Now that we've defined graph convolutions, we can use them to **generalise** the **concept** of **perceptron** to graph networks and define the **graph perceptron**:

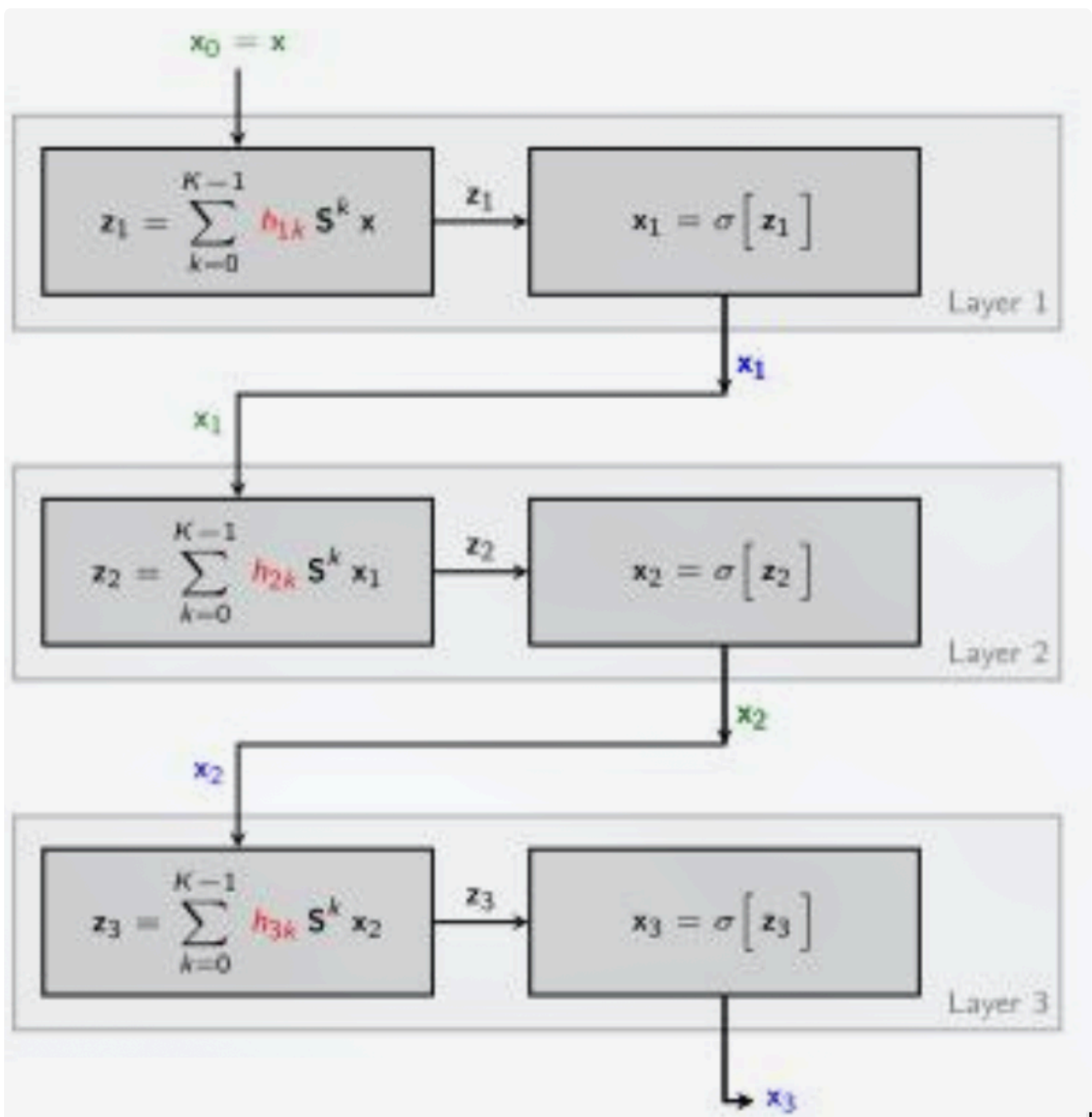
GRAPH PERCEPTRON

Given a graph G , a **GSO** S , a **signal** $\mathbf{x}$ and $K \in \mathbb{R}$, a **graph perceptron** is a **function** that applies an **activation function** to the result of the **graph convolution** between S and $\mathbf{x}$ with **coefficients** $\mathbf{h} \in \mathbb{R}^K$:

$$f_{\mathbf{h}}(\mathbf{x}, S) = \sigma \left(\sum_{k=0}^{K-1} h_k S^k \mathbf{x} \right)$$

Notice that this is indeed very similar to a traditional perceptron: it is still applying an **activation function** to a **weighted sum** of the **inputs**.

We can now build a **graph convolutional network (GCN)** by **stacking multiple graph perceptrons**, as shown below:



NOTE

Notice that a "**layer**" in a GCN is composed of a **single graph perceptron**.

We use the following notation to denote the output of the l -th layer:

$$\mathbf{x}_l = \sigma \left(\sum_{k=0}^{K-1} h_k S^k \mathbf{x}_{l-1} \right)$$

Furthermore, a short hand notation for the **recursive application** over graph perceptrons over **all L layers** is:

$$\hat{\mathbf{y}} = \mathbf{x}_L = f_{\mathcal{H}}(\mathbf{x}, S)$$

Where $\mathcal{H}$ denotes a **set** of L **vectors** of **filter coefficients** (one filter vector for each layer):

$$\mathcal{H} = \{\mathbf{h}_1, \dots, \mathbf{h}_L\}$$

NOTE: REAL-WORLD PROBLEMS

Now, it is important to notice that, even though in our examples we have discussed **signals** that **assign scalar values** to each **node**, in many **real-world applications**, each **node** is associated with a whole **feature vector** or **tensor** instead of a simple scalar.

In such cases, the **signal** $\mathbf{x}$ becomes a **tensor** and the **diffusion sequence** can be **generalised** quite simply, as the **intuition** behind the process **remains the same**.

TRAINING

To train a **GCN** we need a **dataset** $\mathcal{D}$ of **input signals** $\mathbf{x}_i$, **desired output signals** $\mathbf{y}_i$, and **graph shift operators** S_i of graphs.

That is:

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i, S_i)\}$$

Because all of these are **fixed**, it appears clear that the **parameters of our network** are the **filter coefficients** of each **perceptron**.

The **training process** thus consists in finding the **optimal filter vectors**:

$$\mathcal{H}^* = \underset{\mathcal{H}}{\operatorname{argmin}} \sum_{(\mathbf{x}_i, \mathbf{y}_i, S_i) \in \mathcal{D}} \mathcal{L}(f_{\mathcal{H}}(\mathbf{x}_i, S_i), \mathbf{y}_i)$$

Where $\mathcal{L}$ is the **loss function** of choice.

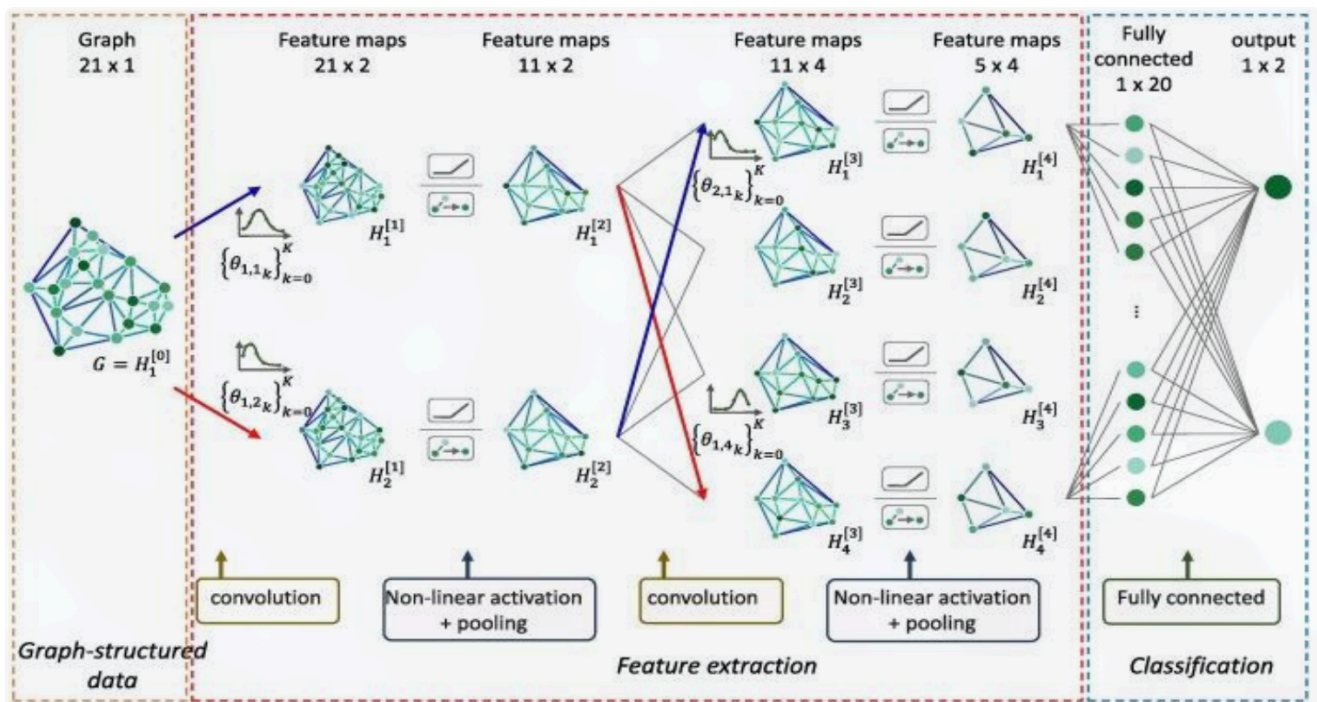
NOTE: DIFFERENT GSOs

It is important to note that our **dataset** should **contain various shift operators** S_i associated with **different graphs** in order for our network to become **more robust** even for **unseen graph representations**.

SOME PRACTICAL EXAMPLES

Now that we know the mathematical details about GCNs, it might be useful to look at **how we can use** this type of networks.

The figure below shows an example GCN architecture for **graph classification**:



Notice that this network **alternates** between **convolution layers** and **pooling layers**:

🔍 OBSERVATION: CONVOLUTIONS AND POOLING

- **Convolution layers** are used to perform **feature extraction**. Notice that by using **multiple filters** we can **produce more features** for each node (e.g. the first layer employs *two filters* to produce *two hidden feature maps*).
- **Pooling layers** are used to **reduce the representation dimensionality** (e.g. after the first pooling layer, each feature map goes from 21 elements to 11). The idea is that, **after convolution has been applied** (and thus **information has diffused**), we can **reduce the graph dimension** while still **preserving information**.

📌 NOTE: POOLING

A **pooling layer** may employ any **graph clustering algorithm** to group sets of nodes together if they **share some desired characteristic**, and then **produce a single output** for **each cluster** (e.g. max pooling outputs the highest value of each cluster).

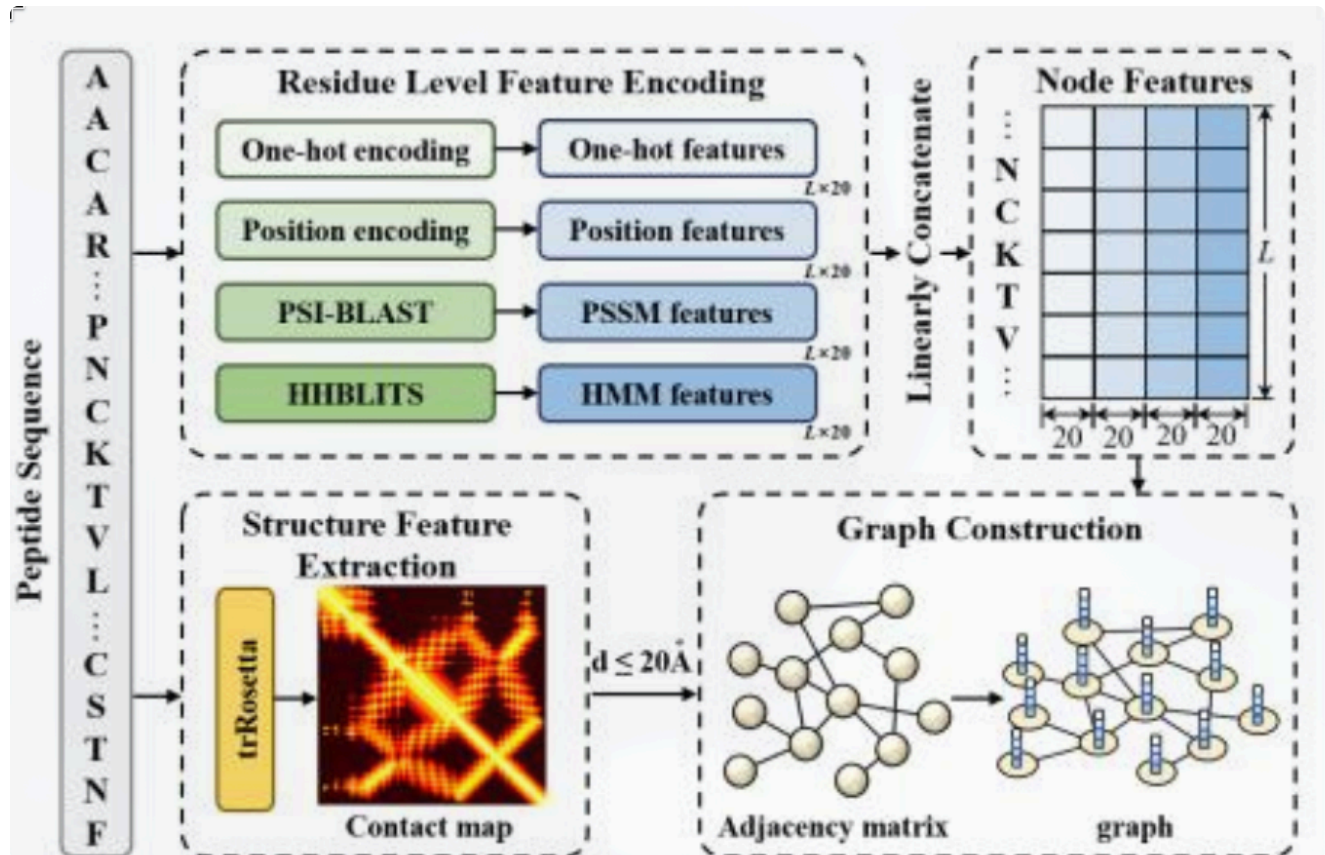
After the last **convolution** and **pooling** operations have been performed, the outputs are passed to a **final fully connected layer** which performs a "readout" operation to **obtain the class prediction** for the input.

A network of this type can be used to **predict properties** of **antimicrobial peptides (AMPs)**

NOTE: peptides are *short chains of amino acids*, and AMPs are *biomolecules* that have *therapeutic functions*.

To predict whether a given **peptide sequence** will be an **effective AMP**, we need a machine learning model with the **ability of understanding its structure**: indeed, **GCNs** are **perfect candidates** for this task.

The figure below illustrates the **pipeline to convert a peptide sequence into a graph**:



Notice that the pipeline consists of **two branches**:

🔍 OBSERVATION: OBTAINING A GRAPH REPRESENTATION

Recall that the **input** a GCN expects is a **pair** $(\mathbf{x}, S)$.

To correctly process a peptide sequence we thus need to extract:

1. **Node features** from each amino acid in the sequence.
2. A **graph shift operator** that **encodes the sequence's structure**.

In the above example, we extract **four features** for **each amino acid** and then **concatenate them** to obtain the **feature vectors** that make up the input **graph signal** $\mathbf{x}$.

To obtain the **graph shift operator** S we can use **trRosetta** (Yang, J., et al.) to **predict the 3D structure** of the AMP from the sequence.

When this 3D structure is obtained, we can use it to extract a **contact map**: a sort of **adjacency matrix** whose **entries** represent **edges** that **connect amino acids** separated by at most 20 angstrom ($1 \text{ \AA} = 0.1 \text{ nm}$).

Finally, we can feed $(\mathbf{x}, S)$ to a **GCN** for **graph classification** like the one presented at the beginning of this paragraph to determine **whether the AMP will be effective or not**.